# 02 · Scheduling-Theorie

Task-Modell, Fristnachweise und kooperativer Dispatcher · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

**Folie** 1 / 50

### 02 · Wer rechnet wann?

- Scheduling-Theorie auf einem Prozessor
- Aus einer Aufgabe werden Jobs mit Zeitparametern
- Regeln auswählen und Fristen begründet prüfen
- Prof. Dr.-Ing. Daniel Klünder · DHBW Stuttgart

**Notizen** 1 / 50

Scheduling bedeutet die Zuteilung von Prozessorzeit. Nachdem Vorlesung 1 einen richtigen und rechtzeitigen Abschluss verlangt hat, fragen wir heute, wie mehrere Aufgaben denselben Kern benutzen. Die Veranstaltung Echtzeitsysteme, kurz EZS, an der Dualen Hochschule Baden-Württemberg, kurz DHBW, verbindet diese Theorie mit einem kleinen Programmvergleich.
Wir trennen drei Ebenen: das Zeitmodell der Aufgaben, die Auswahlregel für bereite Arbeit und deren konkrete Umsetzung. Eine feste Prioritätszahl ist noch kein Fristnachweis. Ein schneller Boardlauf ist umgekehrt kein Beweis für alle zulässigen Ausführungen. Die Rechnungen benutzen ausdrücklich gegebene Lehrannahmen auf einem Prozessor. Im Versuch laufen endliche C-Funktionen auf dem Cortex-M4 des NUCLEO-H755ZI-Q. Ein Echtzeitbetriebssystem müssen Sie dafür noch nicht bedienen können; dessen Threads werden erst in Vorlesung 3 praktisch eingesetzt.

**Folie** 2 / 50

### Vorlesung 1 liefert die Grundlage

| Bekannt | Heute benötigt für |
|---|---|
| Funktion und Rechtzeitigkeit | Abschlussfrist jedes Jobs |
| Rechenbedarf und Antwortzeit | Bedarf versus Wartezeit |
| Polling und Interrupt | kooperative und erzwungene Unterbrechung |
| Messmaximum und sichere Schranke | belastbare Modellparameter |

**Notizen** 2 / 50

Vorlesung 1 hat den Zeitvertrag eingeführt: Beginn, geforderter Abschluss und zulässige Dauer gehören zusammen. Die Antwortzeit reicht von der Freigabe bis zum Abschluss, der eigene Rechenbedarf zählt nur die eigene Arbeit. Polling bedeutet wiederholtes Abfragen; eine Interruptanforderung kann den normalen Kontrollfluss unterbrechen. Ein Messmaximum ist nur der größte bisher beobachtete Wert, keine automatische Garantie für ungesehene Fälle.
Diese Kenntnisse genügen zum Einstieg. Neu sind das systematische Aufgabenmodell und Regeln für mehrere gleichzeitig bereite Aufträge. Der Begriff Task wird zunächst als theoretische Aufgabe verwendet, nicht als bereits eingerichteter Betriebssystem-Thread. Sie brauchen noch keine Kenntnisse über Mutex, Queue oder mehrere zusammenarbeitende Kerne. Eine Mutex wäre eine Zugriffssperre, eine Queue eine Nachrichtenwarteschlange; beide werden heute nicht vorausgesetzt. Bei den Rechnungen ist der Kern eine einzige Rechenressource. Der zweite Kern des Laborboards wird nicht als kostenlose Verdopplung dieser Ressource in die Formeln eingesetzt. Eine spätere Mehrkernanalyse benötigt eigene Annahmen über Verteilung und gegenseitige Störungen.

**Folie** 3 / 50

## Fünf Ergebnisse dieses Termins

1. Task und Job mit $C$, $T$, $D$ und $R$ beschreiben
2. Auslastung von lokalen Fristproblemen unterscheiden
3. Kooperativ, präemptiv und Auswahlregel auseinanderhalten
4. RMS-Schranke, Antwortzeitanalyse und EDF richtig anwenden
5. Einen kooperativen Dispatcher und seine Grenzen erklären

**Notizen** 3 / 50

$C$ ist der eigene Prozessorbedarf, $T$ der Abstand regelmäßiger Freigaben, $D$ die relative Frist und $R$ die vollständige Antwortzeit. Alle vier werden an derselben Aufgabe definiert. Die Auslastung gibt den langfristig benötigten Anteil an Prozessorzeit an, beantwortet aber nicht automatisch jedes kurze Fristfenster. Kooperativ heißt freiwillige Rückkehr beziehungsweise Abgabe, präemptiv eine erzwungene Unterbrechung zugunsten anderer Arbeit.
RMS bedeutet Rate Monotonic Scheduling: Kürzere Perioden erhalten höhere feste Prioritäten. EDF bedeutet Earliest Deadline First: Der bereite Job mit der frühesten absoluten Deadline wird ausgewählt. Die Antwortzeitanalyse, englisch Response-Time Analysis, berechnet unter festgelegten Annahmen eine obere Antwortzeit. Ein Dispatcher setzt die Auswahl um; im Labor bedeutet das einen Funktionsaufruf. Diese Namen sind Orientierung, keine vorausgesetzten Werkzeugkenntnisse. Sie sollen am Ende für jeden Test sagen können, auf welches Modell er passt, was ein bestandener Test beweist und welche Frage nach einem nicht bestandenen hinreichenden Test offenbleibt.

**Folie** 4 / 50

## Vier Schulstunden: rechnen und vergleichen

| Tätigkeit | Minuten |
|---|---|
| Modell und Auswahlregeln · Folien 1–25 | 45 |
| Fristnachweise und Umsetzung · Folien 26–50 | 45 |
| Fünf Papieraufgaben mit Besprechung | 50 |
| Vorbereiteter STM32-Vergleich | 25 |
| Quiz und gemeinsame Klärung | 15 |
| Gesamt | 180 |

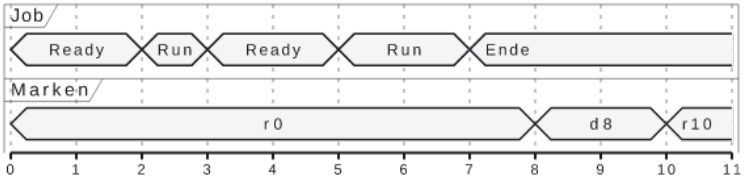
**Notizen** 4 / 50

Vier Schulstunden mit je 45 Minuten ergeben 180 Minuten. Der erste Block schafft die Begriffe und unterscheidet Programmstruktur von Auswahlregel. Der zweite rechnet mehrere Fristnachweise durch und zeigt die Grenze zwischen diesen Beweisen und einem kleinen Anwendungsprogramm. Verständnisfragen sind in die beiden Vortragsblöcke eingebaut. Die fünf Papieraufgaben üben Zeitachsen, Tests und begründete Schlussfolgerungen für eine Klausur.
STM32 ist die Mikrocontrollerfamilie des Laborboards. Die 25 Minuten Programmvergleich setzen das funktionierende Dual-Core-Projekt, also die Firmware für beide Kerne, aus Vorlesung 1 und bereits eingebundene Anwendungsbausteine voraus. Das Konfigurationswerkzeug CubeMX und Visual Studio Code, kurz VS Code, mit der STM32-Erweiterung müssen eingerichtet sein. Erstinstallation, erster Boardstart und vollständige Implementierung von Grund auf passen nicht in diesen Zeitblock. Ohne Board sind die Papieraufgaben vollständig lösbar; der praktische Block kann dem Lesen und Prognostizieren des Codes dienen. Hardwarezeiten werden dadurch nicht zu gemessenen Werten. Das Quiz schließt mit zehn Fragen zu den wichtigen Unterscheidungen.

**Folie** 5 / 50

## Ein Job: eigene Arbeit und ganze Antwort

Lehrmodell · Zeit in ms · Kosten für Verwaltung hier null




**Notizen** 5 / 50

Die horizontale Achse des PlantUML-Timing-Diagramms zeigt Millisekunden. Der Job wird bei $r = 0$

**Notizen, Fortsetzung** 5 / 50

freigegeben; release bedeutet, dass er ab dann rechnen darf. Ready heißt ausführungsbereit. Bei $s = 2$ startet er und läuft bis 3; Run steht für Running, tatsächlich rechnend. Von 3 bis 5 ist er durch andere Arbeit verdrängt und weiterhin Ready. Von 5 bis 7 läuft er erneut. Bei $f = 7$ ist diese Ausführung abgeschlossen. Die Markenspur erinnert an die absolute Deadline $d = 8$ und die nächste Freigabe bei 10, nicht an einen erneuten Start desselben Jobs.
Der eigene Bedarf ist $C = (3 - 2) + (7 - 5) = 3$ ms. Die Startwartezeit beträgt $s - r = 2$ ms, die Unterbrechung ebenfalls 2. Die Antwortzeit ist $R = f - r = 7$ ms. Mit relativer Frist $D = d - r = 8$ ms ist dieser Lauf rechtzeitig und hat eine Millisekunde Reserve. Die Verwaltungskosten sind null angesetzt; reale Kosten müssen passend berücksichtigt werden. Ein neuer Job entsteht durch eine neue Freigabe, nicht durch jede Fortsetzung nach einer Unterbrechung. Genau diese Unterscheidung führt zum Aufgabenmodell.

**Folie** 6 / 50

## Eine Task erzeugt einzelne Jobs

- Aufgabe: $\tau_i$ mit Aufgabennummer $i$
- Ausführung: $J_{i,k}$ mit Jobnummer $k$
- Freigabe, erster Start und Abschluss gehören zu einem Job
- Theoretische Task und Betriebssystem-Thread sind verschiedene Ebenen

**Notizen** 6 / 50

Task heißt Aufgabe. Das Symbol $\tau_i$, gesprochen tau mit Index i, bezeichnet eine wiederkehrende Anforderung, etwa einen Regler oder eine Sensorverarbeitung. Der Index $i$ nummeriert die Aufgaben. $J_{i,k}$ ist der einzelne Job, also die Ausführung Nummer $k$ der Aufgabe $i$. Im Kurs beginnen wir die Jobnummern bei eins; bei synchronen Beispielen ist der erste Job bei Zeit null freigegeben. Eine Aufgabe existiert als Beschreibung über viele Ausführungen hinweg.
Ein Betriebssystem-Thread ist dagegen ein verwalteter Ausführungspfad mit eigenem gespeicherten Kontext. Kontext sind Register- und Stapelinformationen, mit denen ein Programm später fortgesetzt werden kann. Eine einfache C-Funktion kann ebenfalls einen modellierten Job bearbeiten, wenn ihre Freigabe und ihr Abschluss klar definiert werden. Umgekehrt kann ein Thread viele Jobs derselben Aufgabe erledigen. Aus dem Namen einer Funktion folgt daher noch kein Zeitmodell. Die Zerlegung ist nützlich, weil wir für eine Aufgabe gemeinsame Bedarfsschranken und Freigaberegeln angeben, aber jeden Job an seiner eigenen absoluten Deadline beurteilen.

**Folie** 7 / 50

## Der eigene Bedarf wird nach oben begrenzt

- $C_i$: angesetzte obere Grenze der eigenen Prozessorzeit
- WCET: Worst-Case Execution Time im benannten Betriebsmodell
- Warten auf den Prozessor zählt nicht zu $C_i$
- Messmaximum allein begründet keine sichere Schranke

Beispiel: $C_i = 2$ ms sagt noch nichts über alle Wartezeiten.

**Notizen** 7 / 50

$C_i$ ist der im Modell angesetzte obere Bedarf eines Jobs der Aufgabe $i$. CPU heißt Central Processing Unit, Prozessor beziehungsweise Kern. WCET bedeutet Worst-Case Execution Time, die größte eigene Ausführungszeit im benannten Betriebsbereich; für einen Nachweis wird eine begründete obere Schranke verwendet. Die tatsächlich benötigte eigene Zeit eines konkreten Jobs kann kleiner sein. Wartezeit und fremde Unterbrechungsarbeit zählen in unserer Konvention nicht zum eigenen Bedarf.
Eine zwei Millisekunden lange Rechnung kann wegen anderer bereiter Arbeit erst spät starten und daher eine deutlich größere Antwortzeit haben. Ein Marker um eine Funktion misst dagegen verstrichene Zeit und kann fremde Interruptarbeit enthalten. Auch ein größter Wert vieler Messläufe deckt nicht automatisch alle Eingaben und Plattformzustände ab. Compiler, Takt, Speicherzugriffe und weitere Lasten gehören zur Herleitung einer realen Schranke. In den heutigen Zahlenbeispielen sind die Bedarfe ausdrücklich vorgegeben, nicht am Board gemessen. Wir prüfen die Konsequenz dieser Annahmen. Für wiederkehrende Aufgaben müssen wir nun zusätzlich wissen, wann dieser Bedarf erneut freigegeben wird.

**Folie** 8 / 50

## Periode, Frist und Zeitpunkt unterscheiden

| Größe | Bedeutung |
|---|---|
| $T_i$ | Abstand periodischer Freigaben |
| $D_i$ | zulässige Dauer ab einer Freigabe |
| $d_{i,k} = r_{i,k} + D_i$ | absolute Deadline des Jobs |
| $R_{i,k} = f_{i,k} - r_{i,k}$ | Antwortzeit des Jobs |

$C_i = 2, T_i = 10, D_i = 8 \cdot$ Zeiten in ms

**Notizen** 8 / 50

$T_i$ ist die Periode der Aufgabe $i$, also der Abstand ihrer Freigaben. $D_i$ ist die relative Deadline, eine Dauer ab der jeweiligen Freigabe. $r_{i,k}$ ist der Freigabezeitpunkt des Jobs $k$, $f_{i,k}$ dessen Fertigstellungszeitpunkt.

**Notizen, Fortsetzung** 8 / 50

Die absolute Deadline $d_{i,k} = r_{i,k} + D_i$ liegt auf derselben Uhr; $R_{i,k} = f_{i,k} - r_{i,k}$ ist die Antwortzeit dieser einzelnen Ausführung. Für eine allgemeine Garantie benötigen wir eine begründete maximale Antwortzeit $R_i \leq D_i$.
Im Lehrbeispiel hat ein Job höchstens $C_i = 2$ ms eigenen Bedarf und wird alle $T_i = 10$ ms neu freigegeben. Bei Freigabe 10 und relativer Frist 8 liegt seine absolute Deadline bei 18 Millisekunden. Die nächste Freigabe liegt bei 20. Ein Abschluss bei 19 wäre damit bereits zu spät, obwohl der nächste Job noch nicht freigegeben ist. Die Periode misst Freigabeabstände, nicht die vom Scheduler bestimmten Startabstände. Fertigstellung genau auf der Deadline gilt nach unserer Konvention als rechtzeitig. Diese Größen bleiben bei allen folgenden Auswahlregeln dieselben.

**Folie** 9 / 50

## Deadline-Klassen ändern die Testvoraussetzungen

| Klasse | Verhältnis |
|---|---|
| Implizit · implicit | $D_i = T_i$ |
| Beschränkt · constrained | $D_i \leq T_i$ |
| Beliebig · arbitrary | keine solche Einschränkung |

Klassische RMS-Schranke und einfacher EDF-Auslastungstest: $D_i = T_i$.

**Notizen** 9 / 50

Diese Klassifikation betrifft das Verhältnis zwischen relativer Deadline $D_i$ und Freigabeperiode $T_i$, nicht die in Vorlesung 1 eingeführten Klassen hart, fest und weich. Implicit Deadline bedeutet, dass die Abschlussfrist gleich der Periode ist. Constrained Deadline bedeutet eine auf höchstens eine Periode beschränkte Frist; die implizite Klasse ist dabei ein Sonderfall. Arbitrary Deadline lässt das Verhältnis offen, insbesondere kann eine Frist länger als die Periode sein. Dann können sich Jobs derselben Aufgabe überlappen und brauchen eine entsprechende Analyse.
Die klassische Auslastungsschranke für RMS, Rate Monotonic Scheduling mit festen Prioritäten nach Periode, und der einfache Auslastungstest für EDF, Earliest Deadline First nach absoluter Frist, gelten hier für implizite Deadlines. Unsere einfache Antwortzeitanalyse unter festen Prioritäten erlaubt auch beschränkte Deadlines. Ein Satz wie „bei 40 Prozent hält es unter EDF“ ist deshalb ohne das Fristverhältnis unvollständig. Die Zahlen $C = 2, T = 10, D = 8$ Millisekunden gehören zur beschränkten, nicht zur impliziten Klasse. Anschließend betrachten wir, wie die Freigaben selbst entstehen.

**Folie** 10 / 50

## Freigaben können regelmäßig oder ereignisgetrieben sein

| Typ | Annahme über Freigaben |
|---|---|
| Periodisch | fester Abstand $T_i$ |
| Sporadisch | kleinster Abstand $T_i$ |
| Aperiodisch | ohne zugesicherten kleinsten Abstand |

Eine Anforderung braucht eine beschriebene zulässige Eingangslast.

**Notizen** 10 / 50

Eine periodische Aufgabe wird regelmäßig freigegeben, etwa bei 0, 10, 20 und 30 Millisekunden. Ihre Periode $T_i$ ist der feste Abstand. Eine sporadische Aufgabe entsteht durch Ereignisse, für die ein kleinster Abstand zugesichert wird. Bei $T_i = 10$ ms dürfen die Ereignisse beispielsweise bei 0, 13 und 30 eintreffen, aber nicht schon bei 0 und 4. Für bestimmte Fristnachweise begrenzt dieser kleinste Abstand die mögliche Störung ähnlich wie eine Periode.
Aperiodisch bezeichnet hier Ereignisse ohne zugesicherten kleinsten Abstand. Das bedeutet nicht automatisch weich echtzeitkritisch. Für eine harte Zusage muss ihre Last auf andere Weise begrenzt oder über eine vereinbarte Annahme-, Puffer- oder Budgetstrategie geregelt werden. Beliebig viele Ereignisse in beliebig kurzer Zeit kann ein endlicher Kern nicht vollständig bearbeiten. Die Kernbeispiele dieses Termins sind periodisch, synchron bei null gestartet und ohne Freigabeschwankung. Sporadische und aperiodische Typen helfen bei der Einordnung, werden aber nicht stillschweigend durch dieselben einfachen Formeln vollständig behandelt. Die nächste Folie hält die übrigen Modellgrenzen ausdrücklich fest.

**Folie** 11 / 50

## Das Idealmodell macht den Nachweis überschaubar

- Genau ein Prozessor, unabhängige periodische Aufgaben
- Vollständige Präemption, begrenzte Bedarfe $C_i$
- Keine Sperren, kein Freigabejitter, keine Selbstsuspendierung
- Interrupt-, Auswahl- und Wechselkosten zunächst null
- Abschluss auf der Frist ist zulässig

**Notizen** 11 / 50

Unabhängig bedeutet in diesem Modell, dass sich Jobs nur durch die Zuteilung derselben Prozessorzeit behindern. Sie warten nicht auf gemeinsam gehaltene Zugriffssperren. Vollständige Präemption bedeutet, dass ein ausgewählter dringenderer Job den laufenden Job an den modellierten Stellen ohne zusätzliche Sperrphase unterbrechen kann. $C_i$ ist die gegebene obere eigene Ausführungszeit pro Job der Aufgabe $i$.

**Notizen, Fortsetzung** 11 / 50

Freigabejitter wäre eine Schwankung der Freigabezeitpunkte. Selbstsuspendierung wäre ein freiwilliges Warten mitten in einem noch unvollständigen Job. Beides wird in den Grundrechnungen ausgeschlossen. Verwaltungsaufwand, englisch overhead, sowie Interruptarbeit sind zunächst null. Bei echter Firmware dürfen solche Kosten nicht vergessen werden: Sie müssen mit passenden Schranken im Bedarf oder als zusätzliche Störung auftauchen. Ein zweiter Kern, DMA oder Cacheeffekte sind keine kostenlose Erweiterung des Modells. DMA heißt Direct Memory Access, Datenübertragung durch eine eigene Hardwareeinheit; ein Cache ist ein lokaler Zwischenspeicher. Ihre möglichen Störungen brauchen eigene Annahmen. Dieses begrenzte Modell ist ein bewusstes Lehrwerkzeug, dessen Aussagen wir später mit der Realität vergleichen.

**Folie** 12 / 50

## Aus einer Funktion wird ein prüfbares Modell

Lehrannahme für einen Regler:

| Entwurfsangabe | Bedeutung |
|---|---|
| $C = 2\,\text{ms}$ | gegebene Bedarfsschranke |
| $T = 10\,\text{ms}$ | regelmäßige Freigaben |
| $D = 8\,\text{ms}$ | vollständiger Abschluss ab Freigabe |
| Nachweisziel | $R \leq 8\,\text{ms}$ |

Eine Laufzeitpriorität übernimmt diesen Nachweis nicht automatisch.

**Notizen** 12 / 50

Ein Regler berechnet hier einen Stellauftrag. Die Modellangaben sind Lehrannahmen: höchstens $C = 2$ Millisekunden eigene Arbeit, Freigabeperiode $T = 10$ und relative Deadline $D = 8$ Millisekunden. Der Nachweis soll die vollständige maximale Antwortzeit $R$ auf höchstens acht Millisekunden begrenzen. Warten auf andere Aufgaben muss dabei berücksichtigt werden, nicht nur die Rechnung des Reglers.
Zur Laufzeit kann ein Scheduler mit festen Prioritäten lediglich die bereiten Aufgaben und ihre konfigurierten Ränge vergleichen. Er muss dafür die Analyseparameter nicht alle kennen und überwacht sie nicht automatisch. FreeRTOS ist das später im Kurs eingesetzte Echtzeitbetriebssystem; dessen gewöhnliche Prioritätsauswahl ist ebenfalls keine automatische Deadline-Überwachung. Ein nachgewiesener Entwurf wird ungültig, wenn der tatsächliche eigene Bedarf seine angesetzte Grenze überschreitet oder eine ausgeschlossene Wartebedingung hinzukommt. Analyseparameter und Laufzeitverwaltung sind daher verschiedene Ebenen. Als ersten einfachen Blick auf die Gesamtlast berechnen wir den langfristig benötigten Anteil an Prozessorzeit, bevor wir einzelne Fristfenster beurteilen.

**Folie** 13 / 50

### Auslastung beschreibt den langfristigen Bedarf

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

- $n$: Anzahl der Aufgaben
- Summanden und $U$ sind einheitenlos
- Ein Prozessor liefert höchstens einen Anteil von 1
- Bei $U > 1$ passt die dauerhafte Pflichtlast nicht

**Notizen** 13 / 50

$C_i$ ist der obere eigene Rechenbedarf eines Jobs der Aufgabe $i$, $T_i$ deren Periode. Für $n$ Aufgaben summieren wir die Anteile $C_i/T_i$. Ein Bedarf von zwei Millisekunden in jeder Zehn-Millisekunden-Periode beansprucht im langfristigen Modell 0,2 beziehungsweise 20 Prozent der Prozessorzeit. Zähler und Nenner müssen dieselbe Zeiteinheit verwenden; ihr Quotient und die Gesamtgröße $U$ sind einheitenlos. Auf einer CPU, Central Processing Unit beziehungsweise einem Prozessorkern, stehen langfristig höchstens 100 Prozent für die modellierte Arbeit zur Verfügung. Liegt $U$ darüber, ist die dauerhaft vollständig zu erledigende Pflichtlast unmöglich. Liegt $U$ darunter, ist lediglich diese notwendige Kapazitätsbedingung erfüllt. Es ist noch nicht gezeigt, dass die Arbeit vor jeder einzelnen Deadline fertig wird. Verwaltungs- und Interruptkosten sind in diesem Idealmodell null; tatsächlich verfügbare Kapazität kann kleiner sein. Deshalb ist „15 Prozent unbenutzt“ kein frei verfügbarer Puffer in jedem kurzen Zeitfenster. Wir rechnen zunächst eine Summe aus und stellen ihr dann bewusst ein lokales Gegenbeispiel gegenüber.

**Folie** 14 / 50

### Das 75-Prozent-Set als erstes Beispiel

Zeiten in ms · $D_i = T_i$ · erste Freigaben bei null

| Aufgabe | $C_i$ | $T_i$ |
|---|---|---|
| $\tau_1$ | 1 | 4 |
| $\tau_2$ | 2 | 6 |
| $\tau_3$ | 2 | 12 |

$U = \frac{1}{4} + \frac{2}{6} + \frac{2}{12} = \frac{3}{4} = 75\,\%$

**Notizen** 14 / 50

Die drei Aufgaben heißen $\tau_1$, $\tau_2$ und $\tau_3$, tau mit ihrer Aufgabennummer. $C_i$ bezeichnet den oberen eigenen Bedarf, $T_i$ die Freigabeperiode. Ihre relative Deadline $D_i$ ist ausdrücklich gleich der Periode. Alle Größen in der Tabelle sind Millisekunden und didaktische Vorgaben. Die ersten Jobs werden gemeinsam

**Notizen, Fortsetzung** 14 / 50

bei null freigegeben, nicht gemeinsam gestartet.
Die Lastsumme lautet $1/4 + 2/6 + 2/12$. Mit Nenner zwölf wird daraus $3/12 + 4/12 + 2/12 = 9/12 = 3/4 = 0{,}75$, also 75 Prozent. Wir nennen diese konkrete Kombination das 75-Prozent-Set. Der Name steht nicht für jedes beliebige Aufgabensystem gleicher Auslastung. Später vergeben wir feste Prioritäten nach den Perioden und zeichnen die zugehörige Präemption. Die Summe beschreibt zunächst nur den langfristigen Bedarf. Dass die Fristen hier tatsächlich unter der späteren Regel eingehalten werden, muss der passende Test zeigen. Das folgende Beispiel hat sogar eine viel kleinere Summe, aber ein unlösbares kurzes Fristfenster.

**Folie** 15 / 50

## 40 Prozent können trotzdem unmöglich sein

Zwei Jobs werden bei null freigegeben:

- Je $C = 2$ ms und $T = 10$ ms
- Je $D = 3$ ms
- Langfristig $U = 2/10 + 2/10 = 40\,\%$
- Vor $t = 3$ ms sind $4$ ms Arbeit gefordert

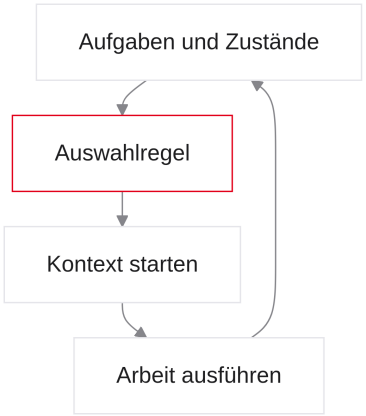
Vier Millisekunden passen nicht in drei Millisekunden eines Kerns.

**Notizen** 15 / 50

Beide Aufgaben haben eigenen Bedarf $C = 2$ Millisekunden und Freigabeperiode $T = 10$ Millisekunden. Ihre relative Deadline $D$ beträgt aber nur drei Millisekunden. Die Auslastung $U$ ist $2/10 + 2/10 = 0{,}4$, also 40 Prozent. Die ersten Jobs sind gleichzeitig bei null bereit und müssen beide bis zum absoluten Zeitpunkt drei abgeschlossen sein.
Ein einzelner Kern kann in diesem Fenster höchstens drei Millisekunden Arbeit leisten; gefordert sind vier. Daher verfehlt mindestens einer der Jobs seine Frist, unabhängig von der Auswahlregel. Eine andere Priorität, eine kürzere Zeitscheibe oder ein später freies Zeitfenster kann diese fehlende Kapazität vor der aktuellen Deadline nicht ersetzen. Das Problem ist grundsätzlich nicht machbar. Zugleich sind die Deadlines beschränkt und nicht implizit, weil $D < T$ gilt. Der später eingeführte einfache Test für EDF, Earliest Deadline First nach frühester absoluter Frist, darf deshalb hier nicht nur aus $U \leq 1$ einen Erfolg ableiten. Eine allgemeine Garantie muss lokale Bedarfsfenster ebenso wie die langfristige Summe berücksichtigen.

**Folie** 16 / 50

## Scheduler entscheidet, Dispatcher setzt um




**Notizen** 16 / 50

Der Scheduler ist die Auswahlkomponente: Er entscheidet, welcher ausführungsbereite Job als Nächstes den Prozessor bekommt. Der Dispatcher setzt diese Entscheidung um. In einem Betriebssystem kann er dazu einen gespeicherten Kontext, also Register und Stapelinformationen, wechseln. Im heutigen Labor ruft er lediglich die gewählte endliche C-Funktion auf. Die Namen benennen Rollen; eine Implementierung kann beide in derselben Funktion verbinden.

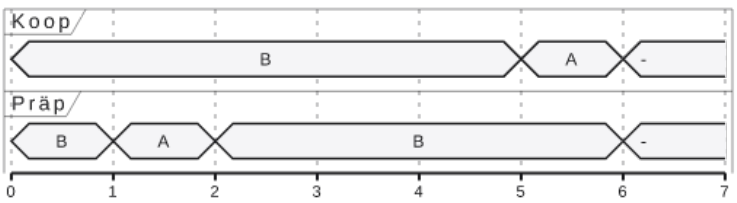
Das Mermaid-Diagramm ist ein Architektur- und Kontrollflussbild. Von oben nach unten werden Verwaltungsdaten gelesen, die rot markierte Auswahl getroffen, der gewählte Kontext gestartet und Arbeit ausgeführt. Der Rückpfeil zeigt eine erneute Entscheidung nach einer zulässigen Rückkehr oder einem passenden Auslöseereignis. Er sagt nichts über eine feste Periode oder sofortige Unterbrechung aus.

Auswahlregel und Umsetzungsmechanismus sind verschieden: Eine Ringauswahl kann kooperativ oder mit erzwungenen Zeitscheiben umgesetzt werden. Eine Regel nach Fristen liefert wiederum noch nicht die technisch richtige Sicherung eines pausierten Threads. Wir unterscheiden deshalb zuerst, wann eine Entscheidung überhaupt wirksam werden darf.

**Folie** 17 / 50

## Kooperativ und präemptiv: anderer Startverzug

Lehrmodell · A kommt bei 1 ms, braucht 1 ms, Deadline bei 3 ms



**Notizen** 17 / 50

Beide Spuren zeigen alternative Ausführungen auf jeweils einem Kern, nicht zwei gleichzeitig benutzte Ressourcen. B braucht fünf Millisekunden eigene Arbeit und beginnt bei null. A wird bei einer Millisekunde freigegeben, benötigt eine Millisekunde und muss bei drei fertig sein. Verwaltungskosten sind null. Koop bezeichnet den kooperativen Ablauf: B gibt erst bei fünf zurück, A läuft von fünf bis sechs und ist drei Millisekunden nach seiner Deadline fertig.
Präp bezeichnet den präemptiven Ablauf. Wenn A gemäß der Auswahlregel dringender ist, unterbricht es B bei eins, rechnet bis zwei und erfüllt die Frist. B setzt danach fort und erreicht seine insgesamt fünf Millisekunden eigene Arbeit erst bei sechs. Der Gedankenstrich kennzeichnet den hier nicht weiter gezeigten Ablauf nach Abschluss. Präemption heißt erzwungene Verdrängung, kooperativ freiwillige Rückgabe. Die obere Spur könnte ebenfalls passen, wenn die Frist von A weiter läge oder B in kurze Schritte zerlegt würde. Das Bild zeigt daher keine allgemeine Unbrauchbarkeit kooperativer Programme. Es zeigt, wie ein langer nicht unterbrechbarer Abschnitt zum Startverzug beitragen kann.

**Folie** 18 / 50

## Auswahlregel und Unterbrechung sind getrennte Achsen

| Entscheidung | Beispiele |
|---|---|
| Wann neu auswählen? | bei Rückkehr oder durch Präemption |
| Wen auswählen? | fester Rang, Ring, früheste absolute Frist |
| Welches Ziel prüfen? | Fristtreue, Wartezeit oder Fairness |

Ein Regelname ersetzt keine Annahme über den Kontrollfluss.

**Notizen** 18 / 50

Eine Auswahlregel sagt, welcher bereite Job bevorzugt wird. Der Unterbrechungsmechanismus sagt, wann die Auswahl wirksam werden kann. Bei festen Prioritäten besitzt eine Aufgabe einen vorher vergebenen Rang. Eine Ringauswahl betrachtet Teilnehmer reihum. Eine Auswahl nach frühester absoluter Deadline vergleicht dagegen konkrete Jobfristen. Jede dieser Ideen muss mit einem zulässigen Kontrollfluss kombiniert werden.
Fairness bedeutet hier, dass bereite Teilnehmer angemessen zum Zug kommen. Fristtreue verlangt rechtzeitige vollständige Abschlüsse. Ein Ring kann unter passenden Bedingungen faire Aufrufreihenfolgen geben und dennoch eine kurze Deadline verletzen. Eine strikte Priorität kann einen wichtigen Job früh bedienen und eine dauerhaft bereite niedrige Aufgabe lange warten lassen. Aus „präemptiv“ folgt weder gleiche Zuteilung noch automatisch Fairness. Aus „Ring“ folgt keine feste Dauer der einzelnen Ausführung. Auch eine theoretisch gute Auswahlregel braucht eine konkrete Implementierung, die Daten und Rückkehrzustand korrekt verwaltet. Diese Unterscheidungen verhindern, dass der spätere kooperative Laborring fälschlich als Umsetzung der präemptiven Fristnachweise gelesen wird.

**Folie** 19 / 50

## Kooperativer Round-Robin verteilt Aufrufe

Drei ständig bereite, zurückkehrende Schritte:

| Schritt | Dauer im Lehrmodell |
|---|---|
| A | 1 ms |
| B | 3 ms |
| C | 1 ms |

Ringfolge A, B, C · A-Starts bei 0, 5, 10, 15 ms

**Notizen** 19 / 50

Round-Robin bedeutet eine Auswahl im Ring beziehungsweise reihum; RR ist die gebräuchliche Abkürzung. In dieser kooperativen Variante ruft der Dispatcher A, dann B, dann C auf und beginnt erneut bei A. Jeder Schritt muss zurückkehren. Die Dauern eins, drei und eins Millisekunden sind Lehrannahmen und schließen hier Verwaltungs- und Interruptkosten aus.
Eine Runde dauert $1 + 3 + 1 = 5$ Millisekunden. Beginnt A bei null, liegen seine nächsten Starts bei fünf, zehn und fünfzehn. A hat nicht deswegen fünf Millisekunden eigenen Rechenbedarf; sein eigener Abschnitt bleibt eine Millisekunde. Der Ring verteilt hier Aufrufe, keine gleichen Zeitmengen. Die Reihenfolge ist fair im begrenzten Sinne, dass jeder ständig bereite Teilnehmer nach endlich langen Nachbarschritten erneut aufgerufen wird. Eine obere Wartegrenze benötigt obere Grenzen aller relevanten Schritte. Kehrt B nicht zurück, kann der Ring überhaupt nicht weiter auswählen, obwohl Interrupts noch laufen können. Einen anderen Mechanismus erhält man, wenn ein Zeitquantum die laufende Ausführung erzwungen beendet.

**Folie** 20 / 50

## Präemptiver Round-Robin verteilt Zeitscheiben

Zwei gleichberechtigte Jobs mit je 2 ms Bedarf · Quantum $q = 1$ ms




**Notizen** 20 / 50

Round-Robin, kurz RR, kann auch präemptiv umgesetzt werden. Ein Quantum $q$ ist dann die höchstens zugeteilte Zeitscheibe, bevor ein anderer gleichberechtigter bereiter Teilnehmer berücksichtigt wird. Im Lehrmodell sind A und B bei null bereit, brauchen jeweils zwei Millisekunden eigene Arbeit und haben denselben Rang. Auswahl- und Wechselkosten sind null; das Quantum beträgt eine Millisekunde.
Die Kernspur zeigt A von null bis eins, B von eins bis zwei, A erneut von zwei bis drei und B von

**Notizen, Fortsetzung** 20 / 50

drei bis vier. Danach sind beide fertig; der Gedankenstrich zeigt keinen weiteren modellierten Job. A endet bei drei, B bei vier. Diese Reihenfolge ist keine Aussage, dass beide jede beliebige Frist erfüllen: Mit gemeinsamer Deadline drei wäre B verspätet. Ein reales System muss beim Umschalten den unterbrochenen Ausführungszustand sichern. Unser Labor tut das nicht und bietet daher nur den kooperativen Aufrufring. In einem priorisierten System kann eine höhere bereite Aufgabe außerdem die Auswahl gleichrangiger Teilnehmer verdrängen. Eine erzwungene Zeitscheibe allein schafft somit keine allgemeine Fristgarantie.

**Folie** 21 / 50

## Ready ist nicht dasselbe wie Blocked

| Zustand | Grund für fehlende oder vorhandene Ausführung |
|---|---|
| Ready | könnte rechnen, bekommt den Kern noch nicht |
| Running | rechnet gerade auf dem Kern |
| Blocked | wartet auf Zeit oder eine sachliche Bedingung |

Auf einem Kern ist höchstens ein Kontext Running.

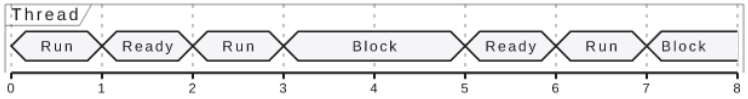
**Notizen** 21 / 50

Ready bedeutet ausführungsbereit: Alle Voraussetzungen zum Rechnen sind erfüllt, aber der Kontext wurde noch nicht ausgewählt oder durch andere Arbeit verdrängt. Running bedeutet tatsächlich laufend. Blocked bedeutet, dass eine notwendige Bedingung fehlt, etwa ein Zeitereignis oder Daten. Ein Kontext ist der gespeicherte Zustand eines Ausführungspfads. Diese Zustandsbegriffe werden besonders in einem RTOS, Real-Time Operating System beziehungsweise Echtzeitbetriebssystem, verwendet, lassen sich aber schon ohne dessen Programmierkenntnisse erklären.
Warten auf die Prozessorzuteilung macht einen Job nicht Blocked. Nach einer Präemption bleibt er Ready, wenn er sofort weiterarbeiten könnte. Wartet ein Thread nach abgeschlossenem Job auf die nächste Freigabe, kann der Thread hingegen Blocked sein, während noch kein neuer Job bereit ist. Ein Jobende und die Existenz eines dauerhaften Threads sind also verschiedene Ebenen. Unser ideales Fristmodell schließt freiwilliges Warten mitten in unvollständigen Jobs aus; Warten zwischen zwei Jobs verletzt diese Annahme nicht. Die folgende Zeitachse verbindet beide Sichtweisen, ohne daraus einen laufenden Betriebssystemversuch zu behaupten.

**Folie** 22 / 50

## Zwischen Jobs warten, innerhalb eines Jobs verdrängt

Lehrzustände eines Threads · Zeit in ms




**Notizen** 22 / 50

Ein Thread ist ein verwalteter Ausführungspfad, der mehrere Jobs derselben Aufgabe bearbeiten kann. Run steht im Diagramm für Running, gerade rechnend; Ready bedeutet ausführungsbereit und Block für Blocked, auf eine Bedingung wartend. Die horizontale Achse zeigt Millisekunden. Der erste Job ist bei null bereit und rechnet von null bis eins. Von eins bis zwei wird er verdrängt, könnte aber weiterrechnen und bleibt daher Ready. Von zwei bis drei rechnet er weiter und endet bei drei.
Dieser erste Job benötigt zwei Millisekunden eigene Arbeit, seine Antwortzeit beträgt drei. Von drei bis fünf wartet der Thread auf die nächste Freigabe und ist Blocked. Das ist keine Selbstsuspendierung eines noch offenen ersten Jobs. Bei fünf entsteht der zweite Job. Er ist bis sechs Ready, rechnet von sechs bis sieben und ist danach abgeschlossen. Seine tatsächliche eigene Arbeit ist hier eine Millisekunde; eine gemeinsame Bedarfsschranke von zwei wäre für beide Beispiele ausreichend. Nach sieben wartet der Thread wieder. Das Diagramm zeigt ein Zustandsmodell, keinen Boardtrace. Der Unterschied erklärt, weshalb ein verdrängter Job in der Analyse weiter zum offenen Arbeitsbedarf gehört.

**Folie** 23 / 50

## Strikte Priorität kann niedrige Aufgaben aushungern

- High und Low bleiben dauerhaft Ready
- High hat den höheren festen Rang
- `yield` gibt freiwillig ab, nimmt High aber nicht aus Ready
- Die nächste Auswahl kann wieder High sein
- Blockieren und geprüfte Lastgrenzen ändern den Fall

**Notizen** 23 / 50

Starvation heißt Aushungern: Eine Aufgabe bleibt bereit, bekommt aber wegen anderer dauerhaft bevorzugter Arbeit keine Prozessorzeit. High und Low sind hier Namen für einen hohen und einen niedrigen festen Rang. Ready bedeutet ausführungsbereit. Bleibt High nach einem freiwilligen yield, also einer Abgabeaufforderung, weiterhin Ready, bevorzugt die strikte Prioritätsregel erneut High. Low muss deshalb nicht als Nächstes laufen. Der dauerhaft bereite Fall erfüllt gerade nicht die begrenzten periodischen Lastannahmen der späteren Grundnachweise.
Blockieren würde High wegen einer Wartebedingung vorübergehend aus der bereiten Menge nehmen und ist etwas anderes als bloße Abgabe. Periodische Freigaben und begrenzte Bedarfe können ebenfalls

**Notizen, Fortsetzung** 23 / 50

verhindern, dass die hohe Aufgabe dauerhaft alle Kapazität verbraucht. Diese Änderungen müssen jedoch zum echten Daten- und Zeitvertrag passen. Präemption allein garantiert keine Fairness. Im kooperativen Programm existiert zusätzlich der andere Fehler, dass eine Funktion nie zurückkehrt; dann bleibt der Dispatcher in ihr, unabhängig von einer später gewünschten Auswahl. Ein Echtzeitbetriebssystem behebt beide Fehler nicht durch seinen Namen. Das Beispiel motiviert die Unterscheidung zwischen der Existenz irgendeines passenden Plans und dem Erfolg einer konkret gewählten Regel.

**Folie** 24 / 50

## Machbarkeit und Erfolg einer Regel unterscheiden

| Frage | Bedeutung |
|---|---|
| Feasibility | Gibt es irgendeinen zulässigen Plan? |
| Schedulability | Hält die gewählte Regel alle Fristen? |

Vier Millisekunden Bedarf vor 3 ms sind für jede Regel unmöglich.
Eine Regel kann aber auch an einem grundsätzlich machbaren Set scheitern.

**Notizen** 24 / 50

Feasibility heißt grundsätzliche Machbarkeit: Es existiert ein zulässiger Ausführungsplan, der alle geforderten Abschlüsse erfüllt. Schedulability bedeutet hier Einplanbarkeit unter einer benannten Auswahlregel und ihren Voraussetzungen. Beim Gegenbeispiel mit zwei Jobs zu je zwei Millisekunden und gemeinsamer Deadline drei gibt es keinen passenden Plan auf einem Kern. Der Mangel liegt in der Kapazität des Fristfensters und kann durch keinen anderen Algorithmus beseitigt werden.
Andere Aufgabenmengen können dagegen unter einer Regel Fristen verletzen und unter einer anderen halten. Das später durchgerechnete Set mit etwa 97 Prozent Last wird diesen Unterschied zeigen. Die Aussage „unter RMS nicht einplanbar“ ist deshalb enger als „überhaupt nicht machbar“. RMS bedeutet Rate Monotonic Scheduling mit festen Prioritäten nach Freigabeperiode. Ein endlicher gezeichneter Erfolgsabschnitt ist außerdem noch kein allgemeiner Beweis für alle späteren Jobs. Ein mathematischer Test kann diesen Nachweis unter seinem Modell übernehmen. Bevor wir solche Tests anwenden, müssen wir wissen, wie ein nicht bestandener Test logisch zu lesen ist.

**Folie** 25 / 50

### Was ein Test wirklich entscheidet

| Testart | Bestanden | Nicht bestanden |
|---|---|---|
| Hinreichend | Erfolg im Modell bewiesen | Frage kann offenbleiben |
| Notwendig | Erfolg kann offenbleiben | Erfolg widerlegt |
| Exakt im Modell | Erfolg bewiesen | Erfolg widerlegt |

Die Modellannahmen gehören zu jeder dieser Aussagen.

**Notizen** 25 / 50

Hinreichend bedeutet, dass eine erfüllte Bedingung den gewünschten Erfolg garantiert. Wird sie nicht erfüllt, folgt daraus noch kein Misserfolg. Notwendig bedeutet, dass ein erfolgreicher Plan die Bedingung erfüllen muss. Ihre Verletzung widerlegt den Erfolg, ihre Erfüllung allein kann die Frage offenlassen. Ein exakter Test ist im benannten Modell sowohl notwendig als auch hinreichend.
Die langfristige Auslastungsbedingung $U \leq 1$ ist zunächst notwendig für dauerhaft vollständig zu erledigende periodische Pflichtlast auf einem Kern; $U$ ist die Summe aus eigenem Bedarf geteilt durch Periode. Das 40-Prozent-Gegenbeispiel zeigt, dass sie bei kürzeren relativen Fristen nicht allgemein hinreichend ist. Die klassische Schranke für RMS, Rate Monotonic Scheduling nach festen periodischen Rängen, ist dagegen hinreichend unter ihren eigenen Voraussetzungen. Eine Überschreitung dieser Schranke bis 100 Prozent erfordert genaueres Prüfen. Diese Logik ist keine sprachliche Feinheit: Sie entscheidet, ob man einen guten Entwurf unnötig verwirft oder einen unbewiesenen Entwurf vorschnell freigibt. Der zweite Vortragsblock wendet sie auf konkrete Regeln an.

**Folie** 26 / 50

### RMS: kürzere Periode, höherer fester Rang

Rate Monotonic Scheduling · Rate $= 1/T_i$

| Aufgabe im 75-%-Set | Periode | Rang |
|---|---|---|
| $\tau_1$ | 4 ms | höchster |
| $\tau_2$ | 6 ms | mittlerer |
| $\tau_3$ | 12 ms | niedrigster |

Die Rangfolge bleibt zwischen den Jobs gleich.

**Notizen** 26 / 50

RMS steht für Rate Monotonic Scheduling. Rate meint die Aktivierungsrate $1/T_i$, wobei $T_i$ die Periode

**Notizen, Fortsetzung** 26 / 50

der Aufgabe $i$ ist. Eine kürzere Periode bedeutet mehr Freigaben pro Zeiteinheit und erhält einen höheren festen Rang. Für die Aufgaben $\tau_1$, $\tau_2$ und $\tau_3$ des 75-Prozent-Sets gilt daher die Reihenfolge vier vor sechs vor zwölf Millisekunden. Die Priorität hängt von der Aufgabe ab und bleibt für deren spätere Jobs gleich.
Das ist eine mathematische Auswahlregel, keine Priorisierung nach Gerätebranche oder gefühlter Wichtigkeit. Ein Kernel mit festen Prioritäten erfüllt diese Regel nur dann, wenn seine konfigurierten Ränge entsprechend vergeben sind. Die numerische Darstellung des Rangs ist implementierungsabhängig; die aus Vorlesung 1 bekannte kleinere Zahl für dringendere Hardwareinterrupts darf nicht ungeprüft auf Threadprioritäten übertragen werden. Unser Theorem setzt außerdem Präemption voraus: Ein neu bereiter Job höheren Rangs kann den niedrigeren verdrängen. Welche Fristgarantie man daraus ableiten darf, hängt von den folgenden Annahmen ab, nicht allein vom Namen RMS.

**Folie** 27 / 50

## Die klassische RMS-Schranke gilt unter Bedingungen

- Unabhängige periodische Aufgaben auf einem Kern
- Feste Ränge nach Periode, vollständige Präemption
- Für jeden Job: $D_i = T_i$ und Bedarf höchstens $C_i$
- Keine Sperren, kein Jitter, keine Selbstsuspendierung
- Verwaltungs- und Interruptkosten im Lehrmodell null

**Notizen** 27 / 50

RMS bedeutet Rate Monotonic Scheduling, die feste Rangfolge nach Aktivierungsrate beziehungsweise Periode. $C_i$ ist die obere eigene Ausführungszeit, $T_i$ die Periode und $D_i$ die relative Deadline der Aufgabe $i$. Für die klassische Auslastungsschranke sind die Deadlines implizit, also $D_i = T_i$. Jobs sind unabhängig und können vollständig nach dem festen Rang verdrängt werden.
Sperren würden zusätzliche Wartezeit auf gemeinsame Ressourcen erzeugen. Freigabejitter wäre eine Schwankung der geplanten Freigaben. Selbstsuspendierung wäre Warten mitten im noch offenen Job. Diese Effekte und Verwaltungs- oder Interruptkosten sind in der Grundrechnung ausgeschlossen. Das bedeutet nicht, dass reale Anwendungen sie ignorieren dürfen. Eine Erweiterung braucht eine passende Analyse und konkrete Schranken, etwa für Sperrzeiten oder zusätzliche Interferenz. Ein gemeinsamer Speicherzugriff ist nicht allein wegen des gemeinsamen Speichers verboten; entscheidend sind seine modellierten Störungen und Abhängigkeiten. Das Board NUCLEO-H755ZI-Q erfüllt die mathematischen Voraussetzungen nicht automatisch, sobald man einen Timer oder mehrere Funktionen programmiert. Unter dem klar begrenzten Modell erhalten wir nun einen schnellen hinreichenden Test.

**Folie** 28 / 50

### Liu und Layland: eine hinreichende Schranke

$$U \leq U_{\mathrm{LL}}(n) = n\left(2^{1/n} - 1\right)$$

| $n$ Aufgaben | Schranke |
|---|---|
| 1 | 100 % |
| 2 | 82,84 % |
| 3 | 77,98 % |
| 4 | 75,68 % |

Bestanden beweist RMS-Erfolg unter den genannten Annahmen.

**Notizen** 28 / 50

$U$ ist die Summe der Bedarfsanteile $C_i/T_i$ und $n$ die positive Anzahl der Aufgaben. $U_{\mathrm{LL}}$ bezeichnet die Schranke von Liu und Layland für Rate Monotonic Scheduling, kurz RMS, im zuvor genannten Idealmodell. Für drei Aufgaben ergibt sich $3(2^{1/3} - 1)$. Die dritte Wurzel von zwei ist ungefähr 1,259921; daraus folgt $3 \cdot 0{,}259921 \approx 0{,}779763$, also gerundet 77,98 Prozent. Die Tabellenwerte sind Prozentwerte, während die Formel Anteile zwischen null und eins benutzt.
Liegt die Auslastung darunter oder genau auf der Schranke, sind die Fristen unter diesen Voraussetzungen nachgewiesen. Die Bedingung ist hinreichend und nicht notwendig. Ihre Überschreitung widerlegt deshalb nicht automatisch RMS. Für sehr viele Aufgaben nähert sich die Schranke $\ln 2 \approx 0{,}693147$, dem natürlichen Logarithmus von zwei. Diese etwa 69,31 Prozent sind kein maximales Lastniveau, das RMS jemals bewältigen könnte. Bestimmte Periodenverhältnisse erlauben mehr. Zuerst wenden wir die einfache Schranke auf den vertrauten Satz mit drei Aufgaben an.

**Folie** 29 / 50

### 75 Prozent: die einfache Schranke entscheidet

Für drei Aufgaben und das gegebene Set:

- $U = 75\,\%$
- $U_{\mathrm{LL}}(3) \approx 77{,}98\,\%$
- Daher unter RMS im Idealmodell nachgewiesen
- Ein anderes Auswahlverfahren braucht seinen eigenen Nachweis

Die Zeichnung illustriert den Test, sie ersetzt ihn nicht.

**Notizen** 29 / 50

Das 75-Prozent-Set besitzt die Bedarfe $C = (1, 2, 2)$ und Perioden $T = (4, 6, 12)$ Millisekunden; die relative Deadline ist jeweils gleich der Periode. $U$ ist die Auslastung und $U_{\mathrm{LL}}(3)$ die Liu-und-Layland-

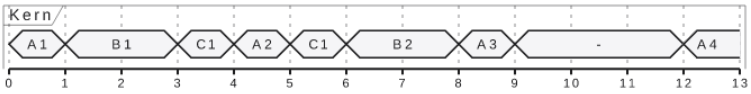
**Notizen, Fortsetzung** 29 / 50

Schranke für drei Aufgaben. Rate Monotonic Scheduling, kurz RMS, ordnet die Ränge nach den Perioden. Weil $0{,}75 \leq 0{,}779763\ldots$ gilt, ist dieser konkrete Satz im unabhängigen vollständig präemptiven Modell nachgewiesen.
Diese Aussage umfasst die periodischen Jobs unter den Modellannahmen, nicht nur ein paar gezeichnete Ausführungen. Die folgende Zeitachse macht dennoch sichtbar, wie ein niedrigerer Job unterbrochen und fortgesetzt wird. Ein häufiges Missverständnis wäre, den gleichen Prozentwert anschließend auf einen kooperativen Ring zu übertragen. Dessen lange nicht unterbrochene Schritte erfüllen die Voraussetzung der vollständigen Präemption gerade nicht. Ebenso wäre eine unmodellierte Interrupt- oder Buslast kein kostenloser Teil der 25 Prozent Restkapazität. Der Test und seine Voraussetzungen gehören zusammen. Die Zeichnung wird deshalb ausdrücklich als Lehrablauf ohne Kosten, nicht als Messung einer Firmware, gelesen.

**Folie** 30 / 50

## Präemptives RMS im 75-Prozent-Set

Lehrablauf in ms · A: $\tau_1$, B: $\tau_2$, C: $\tau_3$ · Ziffer: Jobnummer




**Notizen** 30 / 50

RMS bedeutet Rate Monotonic Scheduling, die feste Priorität nach Periode. Im Timing-Diagramm steht A für $\tau_1$ mit $C = 1, T = D = 4$, B für $\tau_2$ mit $C = 2, T = D = 6$ und C für $\tau_3$ mit $C = 2$, $T = D = 12$ Millisekunden. $C$ ist eigener Bedarf, $T$ Freigabeperiode und $D$ relative Frist. Die Ziffer hinter dem Buchstaben zählt die Jobs; die horizontale Achse zeigt Millisekunden. Alle ersten Jobs sind bei null bereit.
Die halboffenen Laufintervalle sind 0–1 A1, 1–3 B1, 3–4 C1, 4–5 A2 und 5–6 C1. Bei vier verdrängt der neue höhere Job A2 den noch offenen C1. C1 hat am Ende insgesamt zwei Millisekunden gerechnet, aber seine Antwortzeit ab Freigabe null ist sechs. Danach folgen 6–8 B2 und 8–9 A3. Der Gedankenstrich von 9 bis 12 bedeutet unbenutzte modellierte Kapazität. Bei zwölf beginnt A4 aus der nächsten Freigabegruppe und läuft bis dreizehn. Abschluss und neue Freigabe an derselben Grenze werden sauber getrennt: Bei sechs ist C1 schon fertig, bevor B2 den Kern bekommt. Das Bild illustriert die Unterbrechung und den vorherigen Nachweis.

**Folie** 31 / 50

### Der kritische Freigabezeitpunkt

Im unabhängigen präemptiven Modell mit festen Rängen:

- Ein Job kommt gemeinsam mit allen höher priorisierten Aufgaben
- Dadurch entsteht sein kritisches Störungsfenster
- Kein gemeinsamer Prozessorstart: nur einer kann rechnen
- Sperren, Freigabejitter oder Selbstsuspendierung brauchen andere Analysen

**Notizen** 31 / 50

Critical Instant heißt kritischer Freigabezeitpunkt. Für das hier betrachtete feste Prioritätsmodell mit unabhängigen vollständig präemptiven Jobs und beschränkten Deadlines erzeugt die gemeinsame Freigabe mit allen höher priorisierten Aufgaben die maximale Antwortzeit. Die höher priorisierten Aufgaben werden anschließend so früh wie ihre Perioden erlauben erneut freigegeben. Eine gemeinsame Freigabe bedeutet mehrere bereite Jobs und keinesfalls gleichzeitige Ausführung auf einem Kern.
Im 75-Prozent-Set liegt dieser Zeitpunkt bei null. Die Antwortzeiten der Aufgaben sind unter RMS, Rate Monotonic Scheduling nach Periode, eins, drei und sechs Millisekunden. Eine genaue Antwortzeitanalyse kann diese Zahlen herleiten, statt sie nur aus dem einen Ablauf abzulesen. Das Theorem gilt unter seinen Annahmen; zusätzliche Sperrwartezeit, Freigabeschwankungen oder ein freiwilliges Warten mitten im Job können andere kritische Situationen erzeugen. Die Regel „gemeinsam starten reicht immer als Test“ darf deshalb nicht auf beliebige Programme übertragen werden. Nun benutzen wir diesen kritischen Fall, um den störenden Arbeitsbedarf in einem wachsenden Antwortzeitfenster zu zählen.

**Folie** 32 / 50

### Antwortzeit iterativ berechnen

Feste Prioritäten · $D_i \leq T_i$ · Grundmodell

$$R_i^{(0)} = C_i$$

$$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j$$

Stopp bei unverändertem Wert oder Überschreitung von $D_i$.

**Notizen** 32 / 50

RTA bedeutet Response-Time Analysis, Antwortzeitanalyse. $i$ bezeichnet die untersuchte Aufgabe, $C_i$ ihren oberen eigenen Bedarf und $D_i$ ihre relative Deadline und $T_i$ ihre Periode. $hp(i)$ ist die Menge der Aufgaben mit höherer Priorität, englisch higher priority. $T_j$ ist die Periode und $C_j$ der Bedarf eines

**Notizen, Fortsetzung** 32 / 50

solchen Störers $j$. Der Index $k$ nummeriert hier Rechenschritte der Iteration, nicht Jobfreigaben. $R_i^{(k)}$ ist das bisher angenommene Antwortzeitfenster.
Wir beginnen mit der eigenen Arbeit $R_i^{(0)} = C_i$. In einem Fenster dieser Länge können höher priorisierte Jobs dazwischenrechnen. $\lceil R/T_j \rceil$ ist die aufgerundete Anzahl ihrer möglichen Freigaben im kritischen Fenster. Jeder kostet höchstens $C_j$. Die Summe wird zur eigenen Arbeit addiert. Das größere Fenster kann wiederum weitere Freigaben einschließen; deshalb wiederholen wir die Rechnung. Bleibt der Wert unverändert, ist ein Fixpunkt erreicht. Liegt er höchstens auf der Deadline, ist diese Aufgabe im Modell nachgewiesen. Überschreitet ein Wert die Deadline, scheitert sie im betrachteten festen Prioritätsmodell. Die Methode setzt insbesondere $D_i \leq T_i$, keine Sperren, keinen Freigabejitter und keine Selbstsuspendierung voraus. Jeder Summand hat dieselbe Zeiteinheit wie $C_i$.

**Folie** 33 / 50

### Warum wird aufgerundet?

Ein höher priorisierter Störer mit $T_j = 5$ ms:

| Fenster $R$ | Relevante Freigaben | $\lceil R/T_j \rceil$ |
|---|---|---|
| 2 ms | 0 | 1 |
| 5 ms | 0 | 1 |
| 6 ms | 0, 5 | 2 |

Das Fenster ist halboffen: $[0, R)$.

**Notizen** 33 / 50

$R$ bezeichnet die angenommene Antwortdauer des untersuchten Jobs; $T_j$ ist der Freigabeabstand des höher priorisierten Störers $j$. Die Zeichen $\lceil\ \rceil$ bedeuten Aufrunden auf die kleinste ganze Zahl, die nicht kleiner als ihr Inhalt ist. Für $R = 2$ Millisekunden und $T_j = 5$ gilt $\lceil 2/5 \rceil = 1$: Bereits der gleichzeitig bei null freigegebene Störjob muss gezählt werden.
Bei $R = 5$ gilt $\lceil 5/5 \rceil = 1$. Eine Freigabe genau bei fünf liegt am rechten, ausgeschlossenen Rand des Fensters $[0, 5)$. Ist der betrachtete Job genau dann fertig, verzögert sie ihn nicht rückwirkend. Bei $R = 6$ liegt die Freigabe bei fünf dagegen innerhalb des Fensters; $\lceil 6/5 \rceil = 2$ zählt beide Jobs. Diese Randkonvention passt dazu, dass Abschluss auf der Deadline zulässig ist. Abrunden würde bereits bei kurzen Fenstern den Störer bei null vergessen. Immer eine weitere Freigabe am rechten Rand zu zählen würde die gegebene Rechnung dagegen unnötig verändern. Das nächste Beispiel zeigt, wie ein Fenster beim Iterieren genau über eine solche Freigabe wächst.

**Folie** 34 / 50

### 85 Prozent: genauer prüfen

Zeiten in ms · $D_i = T_i$ · RMS-Rangfolge $\tau_1 > \tau_2 > \tau_3$

| Aufgabe | $C_i$ | $T_i$ | Antwortzeitfolge |
|---|---|---|---|
| $\tau_1$ | 1 | 4 | $1 \to 1$ |
| $\tau_2$ | 2 | 5 | $2 \to 3 \to 3$ |
| $\tau_3$ | 2 | 10 | noch zu berechnen |

$U = 1/4 + 2/5 + 2/10 = 85\,\%$ · einfache Schranke unentschieden

**Notizen** 34 / 50

Dieses Set ist bewusst anders als das 75-Prozent-Set. $C_i$ ist der eigene Bedarf, $T_i$ die Periode und $D_i = T_i$ die relative Deadline der Aufgabe $i$. Die Auslastung beträgt $U = 1/4 + 2/5 + 2/10 = 0{,}25 + 0{,}4 + 0{,}2 = 0{,}85$. RMS, Rate Monotonic Scheduling, vergibt die Rangfolge nach vier, fünf und zehn Millisekunden. Die Liu-und-Layland-Schranke für drei Aufgaben von etwa 77,98 Prozent ist überschritten und liefert deshalb keine Entscheidung.
Die RTA, Response-Time Analysis beziehungsweise Antwortzeitanalyse, prüft die Aufgaben einzeln. $\tau_1$ hat keinen höheren Störer; ihr Fixpunkt ist $R_1 = C_1 = 1\,\text{ms} \le 4$. Für $\tau_2$ startet das Fenster bei zwei. Die nächste Rechnung ergibt $2 + \lceil 2/4 \rceil \cdot 1 = 3$. Danach ergibt $2 + \lceil 3/4 \rceil \cdot 1 = 3$ wieder denselben Wert. Somit ist $R_2 = 3\,\text{ms} \le 5$. Beide Fristen halten im Modell. Die dritte Aufgabe wird von beiden höheren Aufgaben beeinflusst. Ihre Rechnung benötigt mehrere Schritte, weil das wachsende Fenster zusätzliche Freigaben bei vier und fünf einschließt.

**Folie** 35 / 50

### Die dritte Antwortzeit wächst bis acht

Für $\tau_3$: $C_3 = 2$, Störer $(C_1, T_1) = (1, 4)$ und $(C_2, T_2) = (2, 5)$ · ms

| Altes $R$ | Jobs von $\tau_1$ | Jobs von $\tau_2$ | Neues $R$ |
|---|---|---|---|
| 2 | 1 | 1 | 5 |
| 5 | 2 | 1 | 6 |
| 6 | 2 | 2 | 8 |
| 8 | 2 | 2 | 8 |

Fixpunkt $R_3 = 8\,\text{ms} \le D_3 = 10\,\text{ms}$.

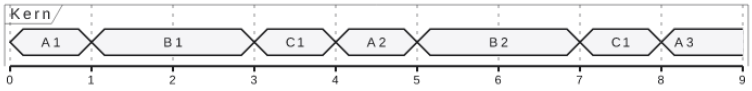
**Notizen** 35 / 50

$C_3 = 2$ Millisekunden ist die eigene Arbeit von Aufgabe $\tau_3$. Die höher priorisierten Störer haben Bedarf $C_1 = 1$, Periode $T_1 = 4$, sowie Bedarf $C_2 = 2$, Periode $T_2 = 5$ Millisekunden. $R$ bezeichnet das Antwortzeitfenster. Die beiden mittleren Tabellenspalten zählen aufgerundet die Störjobs, nicht deren Millisekunden.
Wir starten mit $R = 2$. Daraus ergibt sich $2 + \lceil 2/4 \rceil \cdot 1 + \lceil 2/5 \rceil \cdot 2 = 2 + 1 + 2 = 5$. Mit fünf als Fenster erhält man $2 + 2 \cdot 1 + 1 \cdot 2 = 6$, weil $\lceil 5/4 \rceil = 2$ und $\lceil 5/5 \rceil = 1$. Mit sechs ergibt sich $2 + 2 \cdot 1 + 2 \cdot 2 = 8$. Das Fenster acht zählt erneut zwei Jobs jeder höheren Aufgabe und liefert wieder acht. Der Fixpunkt ist somit $R_3 = 8$ ms, kleiner als die relative Deadline $D_3 = 10$ ms. Die eigene Arbeit ist in den acht bereits enthalten und wird nicht nochmals addiert. Alle drei Aufgaben des konkreten Sets sind nun unter RMS, Rate Monotonic Scheduling, nachgewiesen. Daraus folgt keine allgemeine Sicherheit beliebiger Sets mit 85 Prozent Auslastung; die Perioden und Bedarfe tragen die konkrete Rechnung.

**Folie** 36 / 50

## Der kritische Ablauf bestätigt die Rechnung

85-Prozent-Set · A: $\tau_1$, B: $\tau_2$, C: $\tau_3$ · Zeit in ms




**Notizen** 36 / 50

Die horizontale Millisekundenachse und die Kernspur zeigen das 85-Prozent-Set unter RMS, Rate Monotonic Scheduling nach Periode. A bezeichnet $\tau_1$ mit Bedarf eins und Periode vier, B $\tau_2$ mit Bedarf zwei und Periode fünf, C $\tau_3$ mit Bedarf zwei und Periode zehn Millisekunden. Die relative Deadline ist jeweils gleich der Periode; die Ziffer zählt den Job. Die ersten Jobs sind bei null freigegeben, alle Verwaltungskosten sind null.
Von null bis eins läuft A1, von eins bis drei B1 und von drei bis vier das erste Stück C1. Der bei vier freigegebene A2 verdrängt C1 und läuft bis fünf. Bei fünf wird B2 bereit und hat immer noch höheren Rang als C1; B2 läuft bis sieben. Erst dann setzt C1 von sieben bis acht fort. Seine eigene Arbeit beträgt $C_3 = (4-3) + (8-7) = 2$ ms, seine Antwortzeit $R_3 = 8 - 0 = 8$ ms. A3 wird bei acht freigegeben und läuft bis neun, nachdem C1 bereits fertig ist. Das Bild erklärt die in der Antwortzeitanalyse gezählten Störer. Der allgemeine Fristnachweis stammt aus der Analyse unter den Annahmen, nicht allein aus dieser endlichen Zeichnung.

**Folie** 37 / 50

### Über der Schranke heißt nicht gescheitert

Harmonisches Lehrset · $D_i = T_i$:

| Aufgabe | $C_i$ | $T_i$ |
|---|---|---|
| $\tau_1$ | 2 ms | 4 ms |
| $\tau_2$ | 4 ms | 8 ms |

$U = 100\,\% \cdot R_1 = 2\,\text{ms} \cdot R_2 : 4 \to 6 \to 8 \to 8\,\text{ms}$
Beide Fristen halten im kostenfreien präemptiven Modell.

**Notizen** 37 / 50

Die Perioden vier und acht Millisekunden sind harmonisch: Die längere ist ein ganzzahliges Vielfaches der kürzeren. $C_i$ bezeichnet den eigenen Bedarf, $T_i$ die Periode und $D_i = T_i$ die relative Frist. Die Auslastung $U = 2/4 + 4/8 = 1$ beträgt 100 Prozent und liegt über der Liu-und-Layland-Schranke für zwei Aufgaben von etwa 82,84 Prozent.
RMS, Rate Monotonic Scheduling, bevorzugt die erste Aufgabe. Ihre Antwortzeit ist $R_1 = 2\,\text{ms} \leq 4$. Für die zweite ergibt die Antwortzeitanalyse $4 + \lceil 4/4 \rceil \cdot 2 = 6$, dann $4 + \lceil 6/4 \rceil \cdot 2 = 8$ und nochmals $4 + \lceil 8/4 \rceil \cdot 2 = 8$. Somit liegt der Fixpunkt $R_2 = 8$ genau auf der Deadline und gilt als rechtzeitig. Der Ablauf 0–2 erster Job von $\tau_1$, 2–4 erster Teil von $\tau_2$, 4–6 zweiter Job von $\tau_1$, 6–8 zweiter Teil von $\tau_2$ illustriert dies. Dieses Gegenbeispiel widerlegt die falsche Deutung der hinreichenden Schranke als allgemeine RMS-Maximallast. Ein reales System mit Zusatzkosten kann diese volle Auslastung nicht unverändert übernehmen; die gegebene Rechnung setzt deren Kosten null.

**Folie** 38 / 50

### EDF vergleicht die absoluten Jobfristen

Earliest Deadline First:

- Unter den bereiten Jobs gewinnt das kleinste $d_{i,k}$
- $d_{i,k} = r_{i,k} + D_i$
- Neue Freigabe kann, muss aber nicht verdrängen
- Bei gleicher Deadline bleibt hier der laufende Job

Die Periode ist kein fester EDF-Prioritätsrang.

**Notizen** 38 / 50

EDF heißt Earliest Deadline First, die Auswahl nach der frühesten absoluten Deadline. $r_{i,k}$ ist die Freigabe des Jobs $k$ von Aufgabe $i$, $D_i$ seine relative Frist und $d_{i,k} = r_{i,k} + D_i$ der absolute Fristzeitpunkt. Aus den ausführungsbereiten Jobs wird der mit dem kleinsten absoluten Zeitpunkt ausgewählt. Eine neu

**Notizen, Fortsetzung** 38 / 50

eintreffende Freigabe verdrängt daher nur dann, wenn ihr Job eine frühere Deadline hat und im Modell Präemption zulässig ist.
Anders als bei festen Rängen kann die relative Reihenfolge zweier Aufgaben von den aktuell offenen Jobs abhängen. Die Auswahl wird bei relevanten Ereignissen wie Freigabe oder Abschluss neu geprüft; EDF bedeutet nicht, dass eine ganze Liste in jeder Millisekunde zwingend neu sortiert wird. Bei gleicher Deadline behalten wir im Beispiel den laufenden Job. Ist noch keiner laufend, verwenden wir eine feste Reihenfolge der Aufgabennummern. Eine solche Tie-Regel macht den Ablauf eindeutig, ist aber nicht die eigentliche Fristpriorität. EDF ist auch nicht auf weiche Echtzeit beschränkt. Ob es harte Fristen garantiert, beantwortet ein passender Test unter den folgenden Voraussetzungen.

**Folie** 39 / 50

### Wann ist der einfache EDF-Test exakt?

Eine CPU · unabhängige vollständig präemptive Aufgaben · $D_i = T_i$

$$U \leq 1 \quad \Longleftrightarrow \quad \text{unter EDF einplanbar}$$

- Begrenzte Bedarfe, keine zusätzlichen Kosten oder Warteursachen
- Nicht unverändert bei $D_i < T_i$
- Nicht durch einen zweiten Kern automatisch erweitert

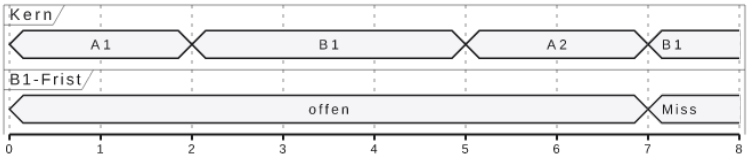
**Notizen** 39 / 50

CPU bedeutet Central Processing Unit, ein Prozessorkern. EDF bedeutet Earliest Deadline First, Auswahl des bereiten Jobs mit frühester absoluter Frist. $U$ ist die Summe der eigenen Bedarfe $C_i$ geteilt durch die Perioden $T_i$. $D_i$ ist die relative Deadline. Im unabhängigen vollständig präemptiven Modell mit $D_i = T_i$ und ohne zusätzliche Kosten ist die Bedingung $U \leq 1$ notwendig und hinreichend. Der Doppelpfeil steht für diese Äquivalenz.
Bei erfüllter Bedingung hält EDF im Modell die Fristen aller periodischen Jobs, nicht nur eines kurzen gezeichneten Ausschnitts. Bei Überschreitung fehlt schon langfristig die erforderliche Kapazität. Kürzere Deadlines fallen aus diesem einfachen Test heraus: Das 40-Prozent-Beispiel mit vier Millisekunden Arbeit vor drei Millisekunden bleibt unmöglich. Für solche Aufgaben sind andere Bedarfsanalysen nötig. Ein zweiter Kern, Sperrzeiten, Freigabejitter, Selbstsuspendierung oder unmodellierte Interruptkosten erfordern ebenfalls eine passende Erweiterung. Wir vergleichen nun dieselbe zulässige Aufgabenmenge mit zwei Regeln; damit wird deutlich, dass das Scheitern von RMS keine allgemeine Unmöglichkeit beweist.

**Folie** 40 / 50

## 97 Prozent: RMS verfehlt eine Frist

$\tau_1 : (C, T, D) = (2, 5, 5), \tau_2 : (4, 7, 7) \cdot \text{ms} \cdot U = 34/35$




**Notizen** 40 / 50

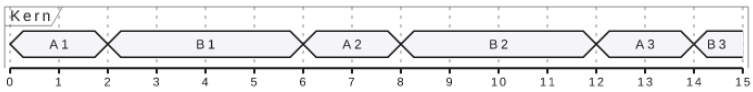
$C$ ist eigener Bedarf, $T$ Periode und $D$ relative Deadline. Die Aufgaben $\tau_1$ und $\tau_2$ haben zwei beziehungsweise vier Millisekunden Bedarf und Perioden fünf beziehungsweise sieben. Die Auslastung ist $U = 2/5 + 4/7 = 14/35 + 20/35 = 34/35 \approx 97{,}14\,\%$. RMS, Rate Monotonic Scheduling, gibt $\tau_1$ den höheren festen Rang. Im Diagramm ist A die erste Aufgabe, B die zweite und die Ziffer die Jobnummer. Die Achse zeigt Millisekunden.

A1 läuft von null bis zwei. B1 erhält von zwei bis fünf nur drei seiner benötigten vier Millisekunden. Bei fünf wird A2 bereit und verdrängt B1 bis sieben. Auf der zweiten Spur erreicht B1 bei sieben seine absolute Deadline, obwohl noch eine Millisekunde fehlt: Miss bedeutet Fristverletzung. Wenn der verspätete Job hier weiterbearbeitet wird, läuft sein letztes Stück von sieben bis acht. Der inzwischen neue B2 wartet; der Nachweis ist bereits am ersten Miss gescheitert. Die Antwortzeitanalyse bestätigt dies: $4 \to 6 \to 8 > 7$. Das Set ist damit nicht unter RMS einplanbar. Seine Last ist aber unter eins; mit einer anderen passenden Regel kann es grundsätzlich machbar sein.

**Folie** 41 / 50

## Dasselbe Set unter EDF

A: $\tau_1$, B: $\tau_2$ · Ziffer: Jobnummer · Zeit in ms




**Notizen** 41 / 50

EDF bedeutet Earliest Deadline First, Auswahl nach frühester absoluter Jobfrist. A steht für $\tau_1$ mit $C = 2$, $T = D = 5$, B für $\tau_2$ mit $C = 4$, $T = D = 7$ Millisekunden. $C$ ist eigener Bedarf, $T$ Periode und $D$ relative Deadline. Die Ziffer zählt den Job; alle ersten Freigaben sind bei null und die Achse zeigt Millisekunden. Die Tie-Regel bei gleicher Frist behält den laufenden Job.

A1 mit Deadline fünf läuft 0–2, B1 mit Deadline sieben 2–6. Obwohl A2 bei fünf bereit wird, hat es erst Deadline zehn und wartet. A2 läuft 6–8. B2, freigegeben bei sieben und mit Deadline vierzehn, läuft 8–12. A3, freigegeben bei zehn und mit Deadline fünfzehn, folgt 12–14. Bei vierzehn wird B3 neu freigegeben und läuft 14–15; dort ist also ausdrücklich kein Leerlauf. Am rechten Rand fünfzehn wird A4 mit Deadline

**Notizen, Fortsetzung** 41 / 50

zwanzig bereit und kann B3 mit Deadline einundzwanzig verdrängen. Die bisherigen Abschlüsse liegen rechtzeitig. Der allgemeine Nachweis ist $U = 34/35 < 1$ bei impliziten Deadlines im Idealmodell; die Zeichnung illustriert ihn und erklärt die anders als bei RMS ausbleibenden Unterbrechungen.

**Folie** 42 / 50

## Neue Freigabe heißt nicht automatisch Präemption

EDF-Auswahl im 97-Prozent-Set:

| Zeitpunkt | Laufender Job | Neuer Job | Entscheidung |
|---|---|---|---|
| 5 ms | $\tau_2/J_1, d = 7$ | $\tau_1/J_2, d = 10$ | laufender bleibt |
| 10 ms | $\tau_2/J_2, d = 14$ | $\tau_1/J_3, d = 15$ | laufender bleibt |

Verglichen werden absolute Fristen, keine frischeren Freigaben.

**Notizen** 42 / 50

EDF heißt Earliest Deadline First. $d$ ist die absolute Deadline, also Freigabezeitpunkt plus relative Frist. $\tau_i/J_k$ bezeichnet in dieser kompakten Tabelle den Job Nummer $k$ der Aufgabe $i$. Bei fünf Millisekunden läuft noch der erste Job von Aufgabe zwei, dessen Deadline sieben ist. Der neue zweite Job von Aufgabe eins hat Freigabe fünf und relative Frist fünf, somit Deadline zehn. Weil sieben früher als zehn liegt, rechnet der laufende Job weiter.
Bei zehn ist die gleiche Unterscheidung erneut wichtig. Der zweite Job von Aufgabe zwei wurde bei sieben freigegeben und hat Deadline vierzehn. Der neue dritte Job der ersten Aufgabe erhält Deadline fünfzehn. Vierzehn ist früher; der laufende Job bleibt bis zu seinem Abschluss bei zwölf. Eine kürzere Periode macht die erste Aufgabe unter RMS, Rate Monotonic Scheduling, dauerhaft höher priorisiert. Unter EDF entscheidet stattdessen die absolute Frist des aktuellen Jobs. Die Tabelle erklärt daher genau, weshalb dieselbe neue Freigabe in beiden Regeln unterschiedliche Unterbrechungen erzeugt. Die eindeutige Wahl hängt nicht von „neu ist immer wichtiger“ ab.

**Folie** 43 / 50

## Bei Überlast muss der Vertrag die Reaktion festlegen

Lehrfall: zwei Aufgaben mit je $C = 3\,\text{ms}$, $T = D = 5\,\text{ms}$

- $U = 6/5 = 120\,\%$
- Schon das erste 5-ms-Fenster verlangt 6 ms Arbeit
- Keine Auswahlregel kann beide aktuellen Fristen erfüllen
- Beenden, Verwerfen oder Budgetbegrenzen ändert den Vertrag

**Notizen** 43 / 50

$C$ ist der eigene Bedarf, $T$ die Periode und $D$ die relative Deadline. Beide ersten Jobs sind bei null freigegeben und brauchen je drei Millisekunden vor dem gemeinsamen Fristzeitpunkt fünf. Die Auslastung $U = 3/5 + 3/5 = 6/5 = 1{,}2$ beträgt 120 Prozent. Schon im ersten Fenster fehlen eine Millisekunde Kapazität und damit mindestens ein vollständiger rechtzeitiger Abschluss. Dauerhaft ist die Pflichtlast ebenfalls zu groß.
EDF, Earliest Deadline First, kann diese Unmöglichkeit ebenso wenig beseitigen wie RMS, Rate Monotonic Scheduling. Unter festgelegter Tie-Regel ist EDF weiterhin eindeutig und nicht plötzlich „chaotisch“; es verfehlt lediglich einen unerfüllbaren Vertrag. Ob verspätete Jobs weiterbearbeitet, verworfen oder durch ein Budget gestoppt werden, beeinflusst nachfolgende Aufträge und muss spezifiziert werden. Bei harten Fristen sind Misses, also Fristverletzungen, nicht zulässig; eine Fehlerreaktion ist trotzdem gesondert zu entwerfen. Bei festen Prioritäten schützt niedrige reine Rechenlast eine höhere Aufgabe nicht vor deren eigener Budgetüberschreitung oder zusätzlichen Sperren. Überlastbehandlung ist eine Entwurfsentscheidung und keine kostenlose Eigenschaft eines Regelnamens.

**Folie** 44 / 50

## Welche Rechnung gehört zu welcher Umsetzung?

| Theoretischer Nachweis | Voraussetzung in der Umsetzung |
|---|---|
| RMS-Schranke | passende feste Ränge und Präemption |
| Antwortzeitanalyse | benannte Störer und Warteursachen |
| EDF-Auslastungstest | implizite Deadlines im Idealmodell |
| Kooperativer Ring | begrenzte zurückkehrende Schritte |

Echte Zusatzkosten und Ressourcenstörungen bleiben zu berücksichtigen.

**Notizen** 44 / 50

RMS heißt Rate Monotonic Scheduling, feste Prioritäten nach Freigabeperiode. EDF heißt Earliest Deadline First, Auswahl nach absoluter Jobfrist. Die Antwortzeitanalyse bestimmt ein Störungsfenster unter einer konkreten festen Rangfolge. Keiner dieser Nachweise ist eine allgemeine Aussage über jeden Code mit denselben Prozentwerten. Die Voraussetzungen müssen in der Umsetzung erhalten oder durch eine passende erweiterte Analyse ersetzt werden.
Im unabhängigen präemptiven festen Prioritätsmodell verdrängt zusätzliche niedrige reine Rechenlast eine für sich nachgewiesene höhere Teilmenge nicht. Eine Sperre, zu lange Maskierung von Interrupts oder höherer eigener Bedarf kann die hohe Aufgabe trotzdem verzögern. Maskierung bedeutet das zeitweise Sperren bestimmter Unterbrechungen. Der kooperative Laborring hat überhaupt keine solche Task-Präemption: Seine Schrittgrenzen bestimmen die nächste Auswahlmöglichkeit. Die Zählschleifen besitzen zudem keine hier behaupteten Millisekunden-WCETs. WCET bedeutet Worst-Case Execution Time, maximale eigene Ausführungszeit im festgelegten Modell. Der praktische Vergleich untersucht Kontrollfluss und beobachtete Zeiten, bestätigt aber nicht automatisch die mathematischen Sets. Damit ist die Brücke zur technischen Umsetzung vorbereitet.

**Folie** 45 / 50

## Tick und PendSV haben unterschiedliche Rollen

| Mechanismus | Rolle im betrachteten Cortex-M-Port |
|---|---|
| SysTick | Zeitbasis und mögliche Auslösung einer Auswahl |
| PendSV | aufgeschobene Ausnahme für Kontextwechsel |
| Handlerpriorität | entscheidet, wann eine Ausnahme laufen darf |

Ein Wechsel kann auch nach einem anderen Interrupt vor dem nächsten Tick erfolgen.

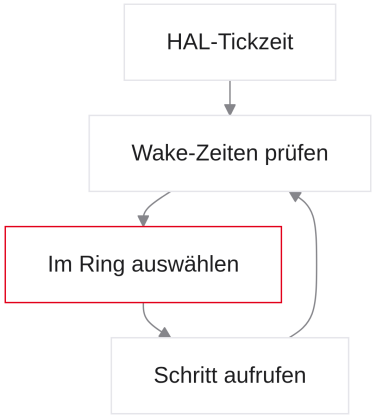
**Notizen** 45 / 50

SysTick ist ein im Cortex-M-Kern integrierter Zeitgeber; ein Tick ist sein periodisches Zeitereignis. PendSV heißt Pendable Service Call, eine softwareseitig anforderbare aufgeschobene Ausnahme. In einem entsprechenden RTOS-Port, also einer Anpassung des Real-Time Operating System an den Kern, kann SysTick Zeit und Wartebedingungen aktualisieren und einen Wechsel anfordern. Auch eine andere ISR, Interrupt Service Routine beziehungsweise Behandlungsroutine, kann einen Thread ausführungsbereit machen und PendSV anfordern.
Pending bedeutet angefordert und noch nicht laufend. Die Priorität der Ausnahme und Maskierung bestimmen, wann PendSV angenommen wird. Im üblichen betrachteten FreeRTOS-Cortex-M-Port wird PendSV niedrig priorisiert, damit erst die aktive dringendere ISR endet. Dann kann der Wechsel bereits vor dem nächsten Tick erfolgen. Eine hohe Threadpriorität verdrängt keinen aktiven Handler allein über ihren Threadrang. PendSV sichert beziehungsweise restauriert den portabhängigen Kontext, zu dem Register und Stapelinformationen gehören. Der in Vorlesung 1 behandelte Hardware-Basisframe allein wählt noch keinen anderen Thread. Unser Labor installiert keinen eigenen PendSV-Port und fordert keinen Threadwechsel an: Der HAL-Tick, Hardware Abstraction Layer beziehungsweise Hardware-Abstraktionsschicht, schreibt hier nur die Millisekundenzeit fort. Die nächste Auswahl folgt erst nach Rückkehr des Schritts.

**Folie** 46 / 50

## Der Labor-Dispatcher bleibt in der Hauptschleife

CubeMX und VS Code mit STM32-Erweiterung · Anwendung auf Cortex-M4




**Notizen** 46 / 50

CubeMX ist das Konfigurations- und Generierungswerkzeug, VS Code steht für Visual Studio Code als Editor mit STM32-Erweiterung. Der Anwendungsbaustein läuft auf dem Cortex-M4, während Cortex-M7 den vorbereiteten gemeinsamen Start beibehält. HAL bedeutet Hardware Abstraction Layer, Hardware-Abstraktionsschicht. Ihre Tickzeit, also fortgeschriebene Millisekundenzeit, liefert den Bezug für Wake-Zeiten; wake heißt Wiederbereitschaft.

Das Mermaid-Bild zeigt Daten- und Aufrufbeziehungen, keine Task-Zeitachse. Die Hauptschleife prüft zuerst, welche gesperrten Einträge ihr Wake-Ziel erreicht haben. Danach sucht sie im Ring einen bereiten Eintrag und ruft dessen endlichen Schritt auf. Erst dessen Rückkehr führt wieder zur Auswahl. Der Pfeil von der Tickzeit führt zur Prüfung, nicht unmittelbar zum Schrittwechsel. Ein Tickinterrupt kann während B weiterlaufen, ohne A aufzurufen. Für den 25-Minuten-Versuch wird das Modul in einem bereits funktionierenden Dual-Core-Projekt bereitgestellt. Der Timer-LED-Versuch aus Vorlesung 1 wird entfernt, und genau ein Kern besitzt den in CubeMX gewählten Markerpin. Die zugehörigen Dateien unter `code/lab02/` enthalten vollständige Anwendungslogik und eine Integrationsanleitung, kein behauptetes fertig getestetes Boardprojekt.

**Folie** 47 / 50

### Verwaltungsdaten statt pausiertem Taskstack

Ausschnitt der C-Datenstruktur im Anwendungsbaustein:

```
typedef struct {
    void (*step)(void);
    TaskState state;
    uint32_t wake_at;
} Lab02_TCB;
```

Ringcursor: nächster Suchbeginn · `running_idx`: nur während eines Aufrufs gültig

**Notizen** 47 / 50

TCB heißt Task Control Block, der Verwaltungsdatensatz eines Ausführungspfads. Der C-Typ `Lab02_TCB` gehört zur Laboranwendung. `struct` fasst die Felder zu einem Datensatz zusammen; `typedef` gibt diesem Typ den verwendeten Namen. `step` ist ein Funktionszeiger: `void (*step)(void)` verweist auf eine Funktion ohne Argumente und ohne Rückgabewert. Dieser Schritt erledigt begrenzte Arbeit und kehrt zurück. `TaskState` ist ein Aufzählungstyp mit den Werten Ready und Blocked, also bereit beziehungsweise auf eine Zeitbedingung wartend. `uint32_t` ist ein vorzeichenloser 32-Bit-Ganzzahltyp. `wake_at` speichert die Tickzeit der vorgesehenen Wiederbereitschaft.
Der Ringcursor ist die Position, ab der die nächste Kandidatensuche beginnt. Nach Auswahl zeigt er auf den Nachfolger. `running_idx` enthält den Index des gerade aufgerufenen Schritts und ist außerhalb des Aufrufs durch einen ungültigen Sonderwert markiert. Das kleine Bereitschaftsfeld braucht deshalb keinen dritten Running-Wert; der aktive Aufruf ist separat bekannt. Die Einträge besitzen keinen eigenen pausierten Registerkontext oder Taskstack. Stack ist Stapelspeicher für Rückkehrinformation und lokale Werte. Ein späterer RTOS-TCB, Real-Time Operating System beziehungsweise Echtzeitbetriebssystem, verwaltet zusätzlich den gesicherten Threadkontext. Im Labor bleibt der Fortschritt zwischen zwei Aufrufen ausdrücklich in Anwendungsdaten.

**Folie** 48 / 50

### Wake-Zeit setzen beendet die Funktion nicht

Ausschnitt innerhalb eines aufgerufenen Schritts:

```
if (!Lab02_SleepAfterStep(10U)) {
    Lab02_Fail();
}
return;
```

Blocked bis zum erreichten Tickziel · erneuter Aufruf erst nach Ringauswahl

**Notizen** 48 / 50

`Lab02_SleepAfterStep` ist die eigene C-Hilfsfunktion des Laborbausteins. Sie nimmt nur innerhalb eines gültigen Schrittaufrufs eine positive Wartezahl kleiner als die halbe 32-Bit-Spanne an. Der Wert `10U` ist eine vorzeichenlose Zehn. Die Funktion setzt `wake_at` auf die aktuelle HAL-Tickzahl plus zehn und das Bereitschaftsfeld auf Blocked, also zeitbedingt nicht bereit. HAL bedeutet Hardware Abstraction Layer. Bei der geprüften 1-kHz-Zeitbasis entspricht ein Tickwertschritt einer Millisekunde. Die Hilfsfunktion liefert bei ungültigem Aufruf null; `Lab02_Fail` behandelt den unerwarteten Fehler.
Das folgende `return` beendet den aufrufenden Schritt selbst. Ein Rücksprung nur aus der Hilfsfunktion würde dagegen den übrigen Schrittcode weiter ausführen. Es wird kein pausierter Stack bewahrt und der Dispatcher nicht rekursiv aufgerufen. Nach Erreichen der Wake-Zeit muss der Dispatcher erst wieder drankommen und den Eintrag im Ring auswählen. Zehn Tickwertschritte garantieren wegen Abtastphase und Auswahlverzug keinen genau zehn Millisekunden späteren physischen Start. Beim Vergleich `Lab02_TimeReached(now, target)` ist `now` die aktuelle Tickzeit und `target` das gespeicherte Wake-Ziel. Er benutzt vorzeichenlose Moduloarithmetik; Warteintervalle und Verspätungen müssen strikt kleiner als $2^{31}$ Millisekunden bleiben. So wird auch der Zählerüberlauf korrekt behandelt. Relatives Warten nach Arbeit erzeugt keine feste Freigabeperiode wie im RMS-Modell.

**Folie** 49 / 50

## Pulsbreite und Startabstand sind verschiedene Größen

Idealer Aufrufring · A und C je 1 ms, B zuerst 1 und dann 3 ms




$w = 1$ ms · Startabstände $P = 3/5$ ms · Spanne $J_P = 2$ ms

**Notizen** 49 / 50

Die WaveDrom-Spur zeigt einen digitalen Marker M während der Arbeit von Schritt A. Eine horizontale Stelle entspricht einer Millisekunde. A und C benötigen im Lehrmodell jeweils eine Millisekunde, B abwechselnd eine und drei. Auswahl-, Aufruf- und Interruptkosten sind null. Die ersten A-Starts liegen bei 0, 3 und 8 Millisekunden; jeder Puls endet eine Millisekunde nach seinem Start. Die Darstellung zeigt damit zwei vollständige Startabstände und den Anfang des dritten Pulses.
$s_k$ und $f_k$ sind Anfang und Ende des Markerpulses Nummer $k$. Die Pulsbreite $w_k = f_k - s_k$ ist Endzeit minus Startzeit der Marke, hier eine Millisekunde. Der Startabstand $P_k = s_{k+1} - s_k$ beträgt zuerst drei, dann fünf Millisekunden. Der beobachtete Peak-to-Peak-Jitter $J_P = \max(P_k) - \min(P_k)$ ist $5 - 3 = 2$ ms. Peak-to-Peak meint die Spanne. Bei konstant langer B-Arbeit wären alle Startabstände fünf und ihre Modellspanne null. Konstante Verlängerung allein ist also noch kein Jitter. Auf dem Board können Interrupts und GPIO-Aufrufaufwand die verstrichene Pulsbreite mit verändern; GPIO bedeutet General-Purpose Input/Output, digitaler Ein-/Ausgang. Die Zählarbeit des Bausteins hat keine zugesicherte Millisekundendauer. Die Modellzahlen gehören daher nicht in ein reales Messprotokoll.

**Folie** 50 / 50

## Ein Nachweis gehört zu seinem Modell

- Aufgaben mit Bedarf, Freigabe und Abschlussfrist beschreiben
- Hinreichende Schranke und exakte Analyse unterscheiden
- 75 % und 85 % unter RMS nachgewiesen
- 97-%-Set: RMS scheitert, ideales EDF hält
- Vorlesung 3: echte Threads mit gespeichertem Kontext

Quellen: Buttazzo · Liu und Layland · ST PM0214, RM0399, UM2408

**Notizen** 50 / 50

RMS bedeutet Rate Monotonic Scheduling mit festen Prioritäten nach Periode; EDF bedeutet Earliest Deadline First nach absoluter Jobfrist. Für den 75-Prozent-Satz reicht die hinreichende Liu-und-Layland-Schranke. Beim 85-Prozent-Satz bleibt sie unentschieden, die Antwortzeitanalyse liefert aber eins, drei und acht Millisekunden als maximale Antwortzeiten. Der Satz mit $U = 34/35$ scheitert unter RMS und ist unter idealem EDF mit impliziten Deadlines nachgewiesen. Die Prozentzahl allein ist also kein allgemeiner RMS-Test.

Der praktische Aufrufring verteilt endliche Schritte ohne eigenen pausierten Kontext. Vorlesung 3 setzt darauf auf und führt FreeRTOS, das verwendete Echtzeitbetriebssystem, mit echten Threads ein. Eine konfigurierte Priorität bleibt auch dort vom Fristnachweis getrennt. Zur Nachbereitung dienen die fünf Papieraufgaben und das Quiz. Theoriequelle ist Giorgio C. Buttazzo, Hard Real-Time Computing Systems. Liu und Layland veröffentlichten die klassische Schranke 1973. STMicroelectronics liefert PM0214 als Programmierhandbuch des Cortex-M4, RM0399 als Peripheriehandbuch des Mikrocontrollers und UM2408 als Boardhandbuch. Cheddar ist der in der Kursübersicht genannte Scheduling-Simulator; eine optionale Simulation muss dieselben Annahmen benutzen und ersetzt einen allgemeinen Satz nicht durch einen kurzen Erfolgsplan. In dieser Überarbeitung werden eigene Modellrechnungen und Hostprüfungen durchgeführt, keine Cheddar- oder Boardläufe behauptet.

Referenzen: Kursliteratur, Liu und Layland (1973), STM32CubeMX, STM32 VS Code Extension. Versionen und Hardwarekonfiguration werden am tatsächlichen Projekt dokumentiert.